

SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE NAME: Database Management Systems

COURSE CODE: CSA0509

COURSE OUTCOME COVERED

CO3: *Analyze relational databases using functional dependencies, normalization (1NF to 5NF), and key constraints to identify redundancy and ensure data integrity. (BL2, BL3)*

Activity Tool : Experiments (High)

Assessment Tool Weightage: 40%

QUESTIONS			Total Marks: 50
Q.No	Question	Bloom's Level	Marks
1	For the relation STUDENT_COURSE(SID, SName, CID, CName, Instructor, Grade), identify all functional dependencies and determine the candidate keys.	BL2 – Understand	5
2	Demonstrate the process of converting an unnormalized relation into 1NF with step-by-step justification and example.	BL3 – Apply	5
3	Given relation R(A,B,C,D,E) with FDs: $A \rightarrow BC$, $BC \rightarrow D$, $D \rightarrow E$. Analyze and decompose it to 2NF eliminating partial dependencies.	BL3 – Apply	5
4	Apply 3NF decomposition on the relation EMPLOYEE(EmpID, EmpName, DeptID, DeptName, ManagerID) with transitive dependencies.	BL3 – Apply	5
5	Verify whether R(A,B,C,D) with FDs: $AB \rightarrow C$, $C \rightarrow D$, $D \rightarrow A$ is in BCNF. If not, decompose it into BCNF.	BL3 – Apply	5
6	Experimentally verify lossless-join decomposition for a given relation using the tabular (Chase) algorithm.	BL3 – Apply	5
7	Identify the multi-valued dependencies in TEACHER(TID, Subject, Hobby) and decompose it into 4NF.	BL3 – Apply	5

8	Demonstrate how join dependencies lead to redundancy. Decompose a given relation to 5NF (Project-Join Normal Form).	BL3 – Apply	5
9	Given a poorly designed database schema for a college, identify all anomalies (insertion, deletion, update) and normalize it to 3NF.	BL2 – Understand	5
10	Compute the closure of attribute sets for the given FDs and determine all candidate keys for the relation.	BL3 – Apply	5

Code of Conduct:

*I **Kadhirezhilan. D [192511228]** certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

RUBRICS FOR CALCULATION:

Criteria	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Concepts	Demonstrates thorough understanding and effectively integrates multiple concepts/technologies	Good understanding with proper integration of most concepts	Basic understanding; limited or partial integration	Poor understanding; unable to integrate concepts
Experimental Design	Well-planned, systematic implementation with optimal solution	Correct implementation with minor issues	Basic implementation with errors or inefficiencies	Incorrect or incomplete implementation
Use of Tools	Effective and advanced use of tools/technologies with proper configuration	Appropriate use of tools with correct execution	Limited or inconsistent use of tools	Incorrect or minimal use of tools
Analysis of Results	Insightful analysis with accurate interpretation and validation of results	Good analysis with reasonable interpretation	Basic analysis with limited insights	Poor or incorrect analysis of results
Documentation	Clear, well-structured documentation with diagrams, code, and results	Organized documentation with necessary details	Basic documentation with missing details	Poor or incomplete documentation

Q1. Functional Dependencies and Candidate Keys

Aim

To identify functional dependencies and determine the candidate key for the relation STUDENT_COURSE.

Relation

STUDENT COURSE (SID, SName, CID, CName, Instructor, Grade)

Functional Dependencies

- $SID \rightarrow SName$
- $CID \rightarrow CName$
- $CID \rightarrow Instructor$
- $(SID, CID) \rightarrow Grade$

Candidate Key

The closure of (SID, CID) is:

$(SID, CID)^+$

$= \{SID, CID\}$

$= \{SID, CID, SName, CName, Instructor, Grade\}$

Therefore:

Candidate Key = (SID, CID)

MySQL Implementation

Step - 01

```
mysql> CREATE TABLE STUDENT_COURSE (  
->     SID INT,  
->     SName VARCHAR(50),  
->     CID INT,  
->     CName VARCHAR(50),  
->     Instructor VARCHAR(50),  
->     Grade VARCHAR(5),  
->     PRIMARY KEY (SID, CID)  
-> );  
Query OK, 0 rows affected (0.22 sec)
```

Step - 02

```
mysql> INSERT INTO STUDENT_COURSE
-> VALUES
-> (101, 'Arun', 501, 'DBMS', 'Kumar', 'A'),
-> (101, 'Arun', 502, 'OS', 'Ravi', 'B+'),
-> (102, 'Bala', 501, 'DBMS', 'Kumar', 'A+'),
-> (103, 'Charan', 503, 'CN', 'Priya', 'B');
Query OK, 4 rows affected (0.04 sec)
Records: 4 Duplicates: 0 Warnings: 0
```

Step - 03

```
mysql> SELECT * FROM STUDENT_COURSE;
+-----+-----+-----+-----+-----+-----+
| SID | SName | CID | CName | Instructor | Grade |
+-----+-----+-----+-----+-----+-----+
| 101 | Arun  | 501 | DBMS  | Kumar      | A      |
| 101 | Arun  | 502 | OS    | Ravi       | B+     |
| 102 | Bala  | 501 | DBMS  | Kumar      | A+     |
| 103 | Charan| 503 | CN    | Priya      | B      |
+-----+-----+-----+-----+-----+-----+
4 rows in set (0.03 sec)
```

Answer to write

Functional Dependencies:

$SID \rightarrow SName$

$CID \rightarrow CName$

$CID \rightarrow Instructor$

$(SID, CID) \rightarrow Grade$

Candidate Key:

(SID, CID)

Step 4 — Prove the key

Try inserting a duplicate:

```
mysql> INSERT INTO STUDENT_COURSE
-> VALUES
-> (101, 'Arun', 501, 'DBMS', 'Kumar', 'A+');
ERROR 1062 (23000): Duplicate entry '101-501' for key 'student_course.PRIMARY'
```

Result

The candidate key is (SID, CID), and the functional dependencies are identified successfully. The functional dependencies were identified successfully and (SID, CID) was determined as the candidate key.

Q2. UNF → 1NF

The question asks you to **demonstrate conversion**, so this is mainly a before/after table experiment.

Step 1 — Create unnormalized table

```
mysql> CREATE TABLE STUDENT_UNF (  
->     SID INT,  
->     SName VARCHAR(50),  
->     Courses VARCHAR(100)  
-> );  
Query OK, 0 rows affected (0.03 sec)
```

Insert

```
mysql> INSERT INTO STUDENT_UNF  
-> VALUES  
-> (101, 'Arun', 'DBMS, OS'),  
-> (102, 'Bala', 'CN, DBMS');  
Query OK, 2 rows affected (0.01 sec)  
Records: 2  Duplicates: 0  Warnings: 0
```

Display

```
mysql> SELECT * FROM STUDENT_UNF;  
+-----+-----+-----+  
| SID  | SName | Courses |  
+-----+-----+-----+  
| 101  | Arun  | DBMS, OS |  
| 102  | Bala  | CN, DBMS |  
+-----+-----+-----+  
2 rows in set (0.00 sec)
```

Step 2 — Convert to 1NF

Create:

```
mysql> CREATE TABLE STUDENT_1NF (  
->     SID INT,  
->     SName VARCHAR(50),  
->     Course VARCHAR(50),  
->     PRIMARY KEY (SID, Course)  
-> );  
Query OK, 0 rows affected (0.03 sec)
```

Insert atomic values:

```
mysql> INSERT INTO STUDENT_1NF
-> VALUES
-> (101, 'Arun', 'DBMS'),
-> (101, 'Arun', 'OS'),
-> (102, 'Bala', 'CN'),
-> (102, 'Bala', 'DBMS');
Query OK, 4 rows affected (0.02 sec)
Records: 4 Duplicates: 0 Warnings: 0
```

Display:

```
mysql> SELECT * FROM STUDENT_1NF;
+-----+-----+-----+
| SID | SName | Course |
+-----+-----+-----+
| 101 | Arun  | DBMS   |
| 101 | Arun  | OS     |
| 102 | Bala  | CN     |
| 102 | Bala  | DBMS   |
+-----+-----+-----+
4 rows in set (0.00 sec)
```

Result

Every cell contains a **single atomic value**.

Therefore:

UNF $\rightarrow$ 1NF

Q3. 2NF

Given:

R (A, B, C, D, E)

$A \rightarrow BC$

$BC \rightarrow D$

$D \rightarrow E$

CREATE:

```
mysql> CREATE TABLE R (  
-> A INT PRIMARY KEY,  
-> B VARCHAR(20),  
-> C VARCHAR(20),  
-> D VARCHAR(20),  
-> E VARCHAR(20)  
-> );  
Query OK, 0 rows affected (0.03 sec)
```

Insert:

```
mysql> INSERT INTO R  
-> VALUES  
-> (1, 'B1', 'C1', 'D1', 'E1'),  
-> (2, 'B2', 'C2', 'D2', 'E2'),  
-> (3, 'B3', 'C3', 'D3', 'E3');  
Query OK, 3 rows affected (0.03 sec)  
Records: 3 Duplicates: 0 Warnings: 0
```

Display:

```
mysql> SELECT * FROM R;  
+---+-----+-----+-----+-----+  
| A | B      | C      | D      | E      |  
+---+-----+-----+-----+-----+  
| 1 | B1     | C1     | D1     | E1     |  
| 2 | B2     | C2     | D2     | E2     |  
| 3 | B3     | C3     | D3     | E3     |  
+---+-----+-----+-----+-----+  
3 rows in set (0.00 sec)
```

Calculation to write

$A^+ = \{A\}$

$A \rightarrow BC$

$A^+ = \{A, B, C\}$

$BC \rightarrow D$

$A^+ = \{A, B, C, D\}$

$D \rightarrow E$

$A^+ = \{A, B, C, D, E\}$

Therefore:

$A^+ = \{A, B, C, D, E\}$

So:

A = Candidate Key

Since the candidate key contains only **one attribute**, partial dependency cannot exist.

Final Answer

R (A, B, C, D, E) is already in 2NF.

No decomposition is required.

Result

The relation satisfies 2NF because there is no partial dependency.

Q4. 3NF Decomposition

Given

EMPLOYEE (EmpID, EmpName, DeptID, DeptName, ManagerID)

Dependencies:

$\text{EmpID} \rightarrow \text{EmpName}, \text{DeptID}$

$\text{DeptID} \rightarrow \text{DeptName}, \text{ManagerID}$

Therefore:

$\text{EmpID} \rightarrow \text{DeptID} \rightarrow \text{DeptName}, \text{ManagerID}$

This is a **transitive dependency**.

Step 1 — Create original table

```
mysql> CREATE TABLE EMPLOYEE (  
->     EmpID INT PRIMARY KEY,  
->     EmpName VARCHAR(50),  
->     DeptID INT,  
->     DeptName VARCHAR(50),  
->     ManagerID INT  
-> );  
Query OK, 0 rows affected (0.03 sec)
```

Insert:

```
mysql> INSERT INTO EMPLOYEE
-> VALUES
-> (101, 'Arun', 10, 'CSE', 501),
-> (102, 'Bala', 10, 'CSE', 501),
-> (103, 'Charan', 20, 'ECE', 502);
Query OK, 3 rows affected (0.03 sec)
Records: 3  Duplicates: 0  Warnings: 0
```

Display:

```
mysql> SELECT * FROM EMPLOYEE;
+-----+-----+-----+-----+-----+
| EmpID | EmpName | DeptID | DeptName | ManagerID |
+-----+-----+-----+-----+-----+
| 101   | Arun    | 10     | CSE      | 501       |
| 102   | Bala    | 10     | CSE      | 501       |
| 103   | Charan  | 20     | ECE      | 502       |
+-----+-----+-----+-----+-----+
3 rows in set (0.00 sec)
```

Step 2 — Decompose into 3NF

Employee table

```
mysql> CREATE TABLE EMPLOYEE_3NF (
->     EmpID INT PRIMARY KEY,
->     EmpName VARCHAR(50),
->     DeptID INT
-> );
Query OK, 0 rows affected (0.04 sec)
```

Department table

```
mysql> CREATE TABLE DEPARTMENT (
->     DeptID INT PRIMARY KEY,
->     DeptName VARCHAR(50),
->     ManagerID INT
-> );
Query OK, 0 rows affected (0.04 sec)
```

Step 3 — Insert data

```
mysql> INSERT INTO EMPLOYEE_3NF
-> VALUES
-> (101, 'Arun', 10),
-> (102, 'Bala', 10),
-> (103, 'Charan', 20);
Query OK, 3 rows affected (0.02 sec)
Records: 3  Duplicates: 0  Warnings: 0
```

```
mysql> INSERT INTO DEPARTMENT
-> VALUES
-> (10, 'CSE', 501),
-> (20, 'ECE', 502);
Query OK, 2 rows affected (0.02 sec)
Records: 2  Duplicates: 0  Warnings: 0
```

Step 4 — Verify using JOIN

```
mysql> SELECT
->     E.EmpID,
->     E.EmpName,
->     E.DeptID,
->     D.DeptName,
->     D.ManagerID
-> FROM EMPLOYEE_3NF E
-> JOIN DEPARTMENT D
-> ON E.DeptID = D.DeptID;
+-----+-----+-----+-----+-----+
| EmpID | EmpName | DeptID | DeptName | ManagerID |
+-----+-----+-----+-----+-----+
| 101   | Arun    | 10     | CSE      | 501        |
| 102   | Bala    | 10     | CSE      | 501        |
| 103   | Charan  | 20     | ECE      | 502        |
+-----+-----+-----+-----+-----+
3 rows in set (0.03 sec)
```

Final decomposition

EMPLOYEE (EmpID, EmpName, DeptID)

DEPARTMENT (DeptID, DeptName, ManagerID)

Result

The transitive dependency is removed and the relation is normalized to 3NF.

Q5. Verify BCNF and Decompose the Relation

Aim

To verify whether the given relation is in **Boyce-Codd Normal Form (BCNF)** and, if not, decompose it into BCNF.

Given Relation

Functional Dependencies:

Step 1: Determine Candidate Keys

Find AB^+

Using $AB \rightarrow C$:

Using $C \rightarrow D$:

Therefore:

Hence, **AB is a candidate key.**

Find BC^+

Using $C \rightarrow D$:

Using $D \rightarrow A$:

Hence, **BC is a candidate key.**

Find BD^+

Using $D \rightarrow A$:

Using $AB \rightarrow C$:

Hence, **BD is also a candidate key.**

Step 2: Check BCNF

A relation is in BCNF if, for every non-trivial functional dependency:

X must be a **superkey**.

Functional Dependency	Determinant	Superkey	BCNF
$AB \rightarrow C$	AB	Yes	Yes
$C \rightarrow D$	C	No	No
$D \rightarrow A$	D	No	No

Therefore:

R is not in BCNF

The dependencies $C \rightarrow D$ and $D \rightarrow A$ violate BCNF.

Step 3: BCNF Decomposition

Using the violating dependency: $C \rightarrow D$

Decompose the relation into: R1 (C, D)

And R2 (A, B, C)

MySQL Implementation

```
mysql> CREATE TABLE R1 (  
->   C INT PRIMARY KEY,  
->   D INT  
-> );  
Query OK, 0 rows affected (0.03 sec)
```

```
mysql> CREATE TABLE R2 (  
->   A INT,  
->   B INT,  
->   C INT,  
->   PRIMARY KEY (A, B)  
-> );  
Query OK, 0 rows affected (0.04 sec)
```

Insert Sample Data

```
mysql> INSERT INTO R1 VALUES  
-> (10, 100),  
-> (20, 200),  
-> (30, 300);  
Query OK, 3 rows affected (0.01 sec)  
Records: 3  Duplicates: 0  Warnings: 0
```

```
mysql> INSERT INTO R2 VALUES  
-> (1, 2, 10),  
-> (3, 4, 20),  
-> (5, 6, 30);  
Query OK, 3 rows affected (0.01 sec)  
Records: 3  Duplicates: 0  Warnings: 0
```

Display the Relations

```
mysql> SELECT * FROM R1;
+----+-----+
| C  | D    |
+----+-----+
| 10 | 100  |
| 20 | 200  |
| 30 | 300  |
+----+-----+
3 rows in set (0.00 sec)
```

```
mysql> SELECT * FROM R2;
+---+---+-----+
| A | B | C    |
+---+---+-----+
| 1 | 2 | 10   |
| 3 | 4 | 20   |
| 5 | 6 | 30   |
+---+---+-----+
3 rows in set (0.00 sec)
```

Verify by Joining the Decomposed Relations

```
mysql> SELECT
->     R2.A,
->     R2.B,
->     R2.C,
->     R1.D
-> FROM R2
-> JOIN R1
-> ON R2.C = R1.C;
+---+---+-----+-----+
| A | B | C    | D    |
+---+---+-----+-----+
| 1 | 2 | 10   | 100  |
| 3 | 4 | 20   | 200  |
| 5 | 6 | 30   | 300  |
+---+---+-----+-----+
3 rows in set (0.00 sec)
```

Result

The relation $R(A,B,C,D)$ is **not in BCNF** because $C \rightarrow D$ and $D \rightarrow A$ have determinants that are not superkeys

Thus, the BCNF decomposition was successfully performed and verified using MySQL.

Q6. Lossless-Join Decomposition Using Chase Algorithm

Aim

To experimentally verify whether a decomposition of a relation is **lossless** using the **tabular (Chase) algorithm**.

Given

Consider the relation: $R(A,B,C)$

Functional Dependency: $A \rightarrow B$

Decomposition: $R_1(A,B)$ and $R_2(A,C)$

Procedure

The common attribute between R_1 and R_2 is A .

Since: $A \rightarrow B$

the value of B can be determined from A . Therefore, the decomposition can reconstruct the original relation without losing information.

MySQL Implementation

```
mysql> CREATE TABLE R1_CHASE (  
->   A INT PRIMARY KEY,  
->   B VARCHAR(20)  
-> );  
Query OK, 0 rows affected (0.27 sec)
```

```
mysql> CREATE TABLE R2_CHASE (  
->   A INT PRIMARY KEY,  
->   C VARCHAR(20)  
-> );  
Query OK, 0 rows affected (0.05 sec)
```

Insert Values

```
mysql> INSERT INTO R1_CHASE VALUES
-> (1, 'B1'),
-> (2, 'B2'),
-> (3, 'B3');
Query OK, 3 rows affected (0.04 sec)
Records: 3  Duplicates: 0  Warnings: 0
```

```
mysql> INSERT INTO R2_CHASE VALUES
-> (1, 'C1'),
-> (2, 'C2'),
-> (3, 'C3');
Query OK, 3 rows affected (0.03 sec)
Records: 3  Duplicates: 0  Warnings: 0
```

Display the Tables

```
mysql> SELECT * FROM R1_CHASE;
+----+-----+
| A  | B    |
+----+-----+
| 1  | B1   |
| 2  | B2   |
| 3  | B3   |
+----+-----+
3 rows in set (0.01 sec)
```

```
mysql> SELECT * FROM R2_CHASE;
+----+-----+
| A  | C    |
+----+-----+
| 1  | C1   |
| 2  | C2   |
| 3  | C3   |
+----+-----+
3 rows in set (0.00 sec)
```

Perform Join

```
mysql> SELECT
->      R1.A,
->      R1.B,
->      R2.C
-> FROM R1_CHASE R1
-> JOIN R2_CHASE R2
-> ON R1.A = R2.A;
+----+-----+-----+
| A  | B    | C    |
+----+-----+-----+
| 1  | B1   | C1   |
| 2  | B2   | C2   |
| 3  | B3   | C3   |
+----+-----+-----+
3 rows in set (0.03 sec)
```

Chase Analysis

The decomposition contains the common attribute A.

Given: $A \rightarrow B$

The matching values of A determine the corresponding values of B. The original relation can therefore be reconstructed by joining R1 and R2.

Result

The decomposition: $R(A,B,C) \rightarrow R1(A,B), R2(A,C)$ is **lossless-join**.

Q7. Identify Multivalued Dependencies and Decompose into 4NF

Aim

To identify the multivalued dependencies in the relation TEACHER(TID, Subject, Hobby) and decompose the relation into **Fourth Normal Form (4NF)**.

Given Relation

Assume that a teacher can have multiple subjects and multiple hobbies independently.

TEACHER(TID,Subject,Hobby)

Multivalued Dependencies

TID $\twoheadrightarrow$ Subject

TID $\twoheadrightarrow$ Hobby

These independent multivalued dependencies cause redundant combinations in the relation.

Step 1: Create the Original Table

```
mysql> CREATE TABLE TEACHER (  
->     TID INT,  
->     Subject VARCHAR(50),  
->     Hobby VARCHAR(50)  
-> );  
Query OK, 0 rows affected (0.06 sec)
```

Step 2: Insert Values

```
mysql> INSERT INTO TEACHER VALUES  
-> (101, 'DBMS', 'Cricket'),  
-> (101, 'DBMS', 'Music'),  
-> (101, 'OS', 'Cricket'),  
-> (101, 'OS', 'Music');  
Query OK, 4 rows affected (0.01 sec)  
Records: 4  Duplicates: 0  Warnings: 0
```

Step 3: Display the Table

```
mysql> SELECT * FROM TEACHER;  
+-----+-----+-----+  
| TID | Subject | Hobby |  
+-----+-----+-----+  
| 101 | DBMS    | Cricket |  
| 101 | DBMS    | Music   |  
| 101 | OS      | Cricket |  
| 101 | OS      | Music   |  
+-----+-----+-----+  
4 rows in set (0.00 sec)
```

The same teacher has multiple subjects and multiple hobbies, resulting in unnecessary combinations.

Step 4: Decompose into 4NF

Create a relation for teacher-subject:

```
mysql> CREATE TABLE TEACHER_SUBJECT (  
->     TID INT,  
->     Subject VARCHAR(50),  
->     PRIMARY KEY (TID, Subject)  
-> );  
Query OK, 0 rows affected (0.04 sec)
```

Create a relation for teacher-hobby:

```
mysql> CREATE TABLE TEACHER_HOBBY (  
->     TID INT,  
->     Hobby VARCHAR(50),  
->     PRIMARY KEY (TID, Hobby)  
-> );  
Query OK, 0 rows affected (0.03 sec)
```

Step 5: Insert Values

```
mysql> INSERT INTO TEACHER_SUBJECT VALUES  
-> (101, 'DBMS'),  
-> (101, 'OS');  
Query OK, 2 rows affected (0.02 sec)  
Records: 2  Duplicates: 0  Warnings: 0
```

```
mysql> INSERT INTO TEACHER_HOBBY VALUES  
-> (101, 'Cricket'),  
-> (101, 'Music');  
Query OK, 2 rows affected (0.01 sec)  
Records: 2  Duplicates: 0  Warnings: 0
```

Step 6: Display the Decomposed Tables

```
mysql> SELECT * FROM TEACHER_SUBJECT;  
+-----+-----+  
| TID | Subject |  
+-----+-----+  
| 101 | DBMS    |  
| 101 | OS      |  
+-----+-----+  
2 rows in set (0.00 sec)
```

```
mysql> SELECT * FROM TEACHER_HOBBY;  
+-----+-----+  
| TID | Hobby   |  
+-----+-----+  
| 101 | Cricket |  
| 101 | Music   |  
+-----+-----+  
2 rows in set (0.00 sec)
```

Decomposition

TEACHER_SUBJECT(TID,Subject)

TEACHER_HOBBY(TID,Hobby)

Result

The multivalued dependencies were identified and the TEACHER relation was successfully decomposed into **4NF**, reducing redundant combinations and improving data organization.

Q8. Join Dependency and 5NF

Aim

To demonstrate how **join dependencies** lead to redundancy and decompose a relation into **Fifth Normal Form (5NF)** or **Project-Join Normal Form (PJ/NF)**.

Experimental Relation

Consider:

SUPPLY(Supplier,Project,Part)

A supplier may be associated with multiple projects and parts. Storing all combinations in one relation can cause redundancy.

Step 1: Create the Original Table

```
mysql> CREATE TABLE SUPPLY (  
->     Supplier VARCHAR(20),  
->     Project VARCHAR(20),  
->     Part VARCHAR(20)  
-> );  
Query OK, 0 rows affected (0.06 sec)
```

Step 2: Insert Values

```
mysql> INSERT INTO SUPPLY VALUES  
-> ('S1', 'P1', 'Part1'),  
-> ('S1', 'P1', 'Part2'),  
-> ('S1', 'P2', 'Part1'),  
-> ('S1', 'P2', 'Part2');  
Query OK, 4 rows affected (0.03 sec)  
Records: 4  Duplicates: 0  Warnings: 0
```

Step 3: Display the Table

```
mysql> SELECT * FROM SUPPLY;
+-----+-----+-----+
| Supplier | Project | Part |
+-----+-----+-----+
| S1       | P1      | Part1 |
| S1       | P1      | Part2 |
| S1       | P2      | Part1 |
| S1       | P2      | Part2 |
+-----+-----+-----+
4 rows in set (0.02 sec)
```

The independent combinations of supplier, project and part can result in redundant tuples.

Step 4: Decompose the Relation

```
mysql> CREATE TABLE SP (
  ->     Supplier VARCHAR(20),
  ->     Project VARCHAR(20),
  ->     PRIMARY KEY (Supplier, Project)
  -> );
Query OK, 0 rows affected (0.06 sec)
```

```
mysql> CREATE TABLE ST (
  ->     Supplier VARCHAR(20),
  ->     Part VARCHAR(20),
  ->     PRIMARY KEY (Supplier, Part)
  -> );
Query OK, 0 rows affected (0.17 sec)
```

```
mysql> CREATE TABLE PT (
  ->     Project VARCHAR(20),
  ->     Part VARCHAR(20),
  ->     PRIMARY KEY (Project, Part)
  -> );
Query OK, 0 rows affected (0.06 sec)
```

Step 5: Insert Values

```
mysql> INSERT INTO SP VALUES
      -> ('S1', 'P1'),
      -> ('S1', 'P2');
Query OK, 2 rows affected (0.03 sec)
Records: 2  Duplicates: 0  Warnings: 0
```

```
mysql> INSERT INTO ST VALUES
      -> ('S1', 'Part1'),
      -> ('S1', 'Part2');
Query OK, 2 rows affected (0.03 sec)
Records: 2  Duplicates: 0  Warnings: 0
```

```
mysql> INSERT INTO PT VALUES
      -> ('P1', 'Part1'),
      -> ('P1', 'Part2'),
      -> ('P2', 'Part1'),
      -> ('P2', 'Part2');
Query OK, 4 rows affected (0.03 sec)
Records: 4  Duplicates: 0  Warnings: 0
```

Step 6: Display the Decomposed Tables

```
mysql> SELECT * FROM SP;
+-----+-----+
| Supplier | Project |
+-----+-----+
| S1       | P1      |
| S1       | P2      |
+-----+-----+
2 rows in set (0.00 sec)
```

```
mysql> SELECT * FROM ST;
+-----+-----+
| Supplier | Part   |
+-----+-----+
| S1       | Part1  |
| S1       | Part2  |
+-----+-----+
2 rows in set (0.00 sec)
```

```
mysql> SELECT * FROM PT;
```

Project	Part
P1	Part1
P1	Part2
P2	Part1
P2	Part2

4 rows in set (0.00 sec)

Step 7: Reconstruct the Relation

```
mysql> SELECT
->     SP.Supplier,
->     SP.Project,
->     ST.Part
-> FROM SP
-> JOIN ST
->     ON SP.Supplier = ST.Supplier
-> JOIN PT
->     ON SP.Project = PT.Project
->     AND ST.Part = PT.Part;
```

Supplier	Project	Part
S1	P1	Part1
S1	P1	Part2
S1	P2	Part1
S1	P2	Part2

4 rows in set (0.03 sec)

Decomposition

SP(Supplier,Project)

ST(Supplier,Part)

PT(Project,Part)

Result

The original relation was decomposed into smaller relations to reduce redundancy caused by join dependencies and demonstrate **5NF (Project-Join Normal Form)**.

Q9. Identify Anomalies and Normalize the College Database to 3NF

Aim

To identify **insertion, deletion, and update anomalies** in a poorly designed college database schema and normalize the relation up to **Third Normal Form (3NF)**.

Poorly Designed Relation

Consider the relation:

COLLEGE (StudentID, StudentName, CourseID, CourseName, InstructorID, InstructorName)

Step 1: Create the Table

```
mysql> CREATE TABLE COLLEGE (  
->     StudentID INT,  
->     StudentName VARCHAR(50),  
->     CourseID INT,  
->     CourseName VARCHAR(50),  
->     InstructorID INT,  
->     InstructorName VARCHAR(50)  
-> );  
Query OK, 0 rows affected (0.09 sec)
```

Step 2: Insert Sample Data

```
mysql> INSERT INTO COLLEGE VALUES  
-> (101, 'Arun', 501, 'DBMS', 201, 'Kumar'),  
-> (102, 'Bala', 501, 'DBMS', 201, 'Kumar'),  
-> (103, 'Charan', 502, 'OS', 202, 'Priya');  
Query OK, 3 rows affected (0.02 sec)  
Records: 3  Duplicates: 0  Warnings: 0
```

Step 3: Display the Table

```
mysql> SELECT * FROM COLLEGE;  
+-----+-----+-----+-----+-----+-----+  
| StudentID | StudentName | CourseID | CourseName | InstructorID | InstructorName |  
+-----+-----+-----+-----+-----+-----+  
| 101 | Arun | 501 | DBMS | 201 | Kumar |  
| 102 | Bala | 501 | DBMS | 201 | Kumar |  
| 103 | Charan | 502 | OS | 202 | Priya |  
+-----+-----+-----+-----+-----+-----+  
3 rows in set (0.01 sec)
```

Anomalies

1. Insertion Anomaly

A new course cannot be inserted easily without providing student information.

2. Update Anomaly

If an instructor's name changes, the same instructor information must be updated in multiple records.

For example:

```
mysql> UPDATE COLLEGE
-> SET InstructorName = 'Dr. Kumar'
-> WHERE InstructorID = 201;
Query OK, 2 rows affected (0.04 sec)
Rows matched: 2  Changed: 2  Warnings: 0
```

3. Deletion Anomaly

If the only student enrolled in a course is deleted, the course and instructor information may also be lost.

Step 4: Normalize to 3NF

The relation is decomposed into the following tables:

STUDENT

```
mysql> CREATE TABLE STUDENT (
->     StudentID INT PRIMARY KEY,
->     StudentName VARCHAR(50)
-> );
Query OK, 0 rows affected (0.07 sec)
```

COURSE

```
mysql> CREATE TABLE COURSE (
->     CourseID INT PRIMARY KEY,
->     CourseName VARCHAR(50),
->     InstructorID INT
-> );
Query OK, 0 rows affected (0.07 sec)
```

INSTRUCTOR

```
mysql> CREATE TABLE INSTRUCTOR (  
->     InstructorID INT PRIMARY KEY,  
->     InstructorName VARCHAR(50)  
-> );  
Query OK, 0 rows affected (0.06 sec)
```

ENROLLMENT

```
mysql> CREATE TABLE ENROLLMENT (  
->     StudentID INT,  
->     CourseID INT,  
->     PRIMARY KEY (StudentID, CourseID)  
-> );  
Query OK, 0 rows affected (0.07 sec)
```

Step 5: Insert Data into Normalized Tables

```
mysql> INSERT INTO STUDENT VALUES  
-> (101, 'Arun'),  
-> (102, 'Bala'),  
-> (103, 'Charan');  
Query OK, 3 rows affected (0.03 sec)  
Records: 3 Duplicates: 0 Warnings: 0
```

```
mysql> INSERT INTO INSTRUCTOR VALUES  
-> (201, 'Kumar'),  
-> (202, 'Priya');  
Query OK, 2 rows affected (0.03 sec)  
Records: 2 Duplicates: 0 Warnings: 0
```

```
mysql> INSERT INTO COURSE VALUES  
-> (501, 'DBMS', 201),  
-> (502, 'OS', 202);  
Query OK, 2 rows affected (0.03 sec)  
Records: 2 Duplicates: 0 Warnings: 0
```

```
mysql> INSERT INTO ENROLLMENT VALUES  
-> (101, 501),  
-> (102, 501),  
-> (103, 502);  
Query OK, 3 rows affected (0.02 sec)  
Records: 3 Duplicates: 0 Warnings: 0
```

Step 6: Display the Tables

```
mysql> SELECT * FROM STUDENT;
+-----+-----+
| StudentID | StudentName |
+-----+-----+
|          101 | Arun        |
|          102 | Bala        |
|          103 | Charan      |
+-----+-----+
3 rows in set (0.00 sec)
```

```
mysql> SELECT * FROM COURSE;
+-----+-----+-----+
| CourseID | CourseName | InstructorID |
+-----+-----+-----+
|          501 | DBMS        |          201 |
|          502 | OS          |          202 |
+-----+-----+-----+
2 rows in set (0.00 sec)
```

```
mysql> SELECT * FROM INSTRUCTOR;
+-----+-----+
| InstructorID | InstructorName |
+-----+-----+
|          201 | Kumar          |
|          202 | Priya          |
+-----+-----+
2 rows in set (0.00 sec)
```

```
mysql> SELECT * FROM ENROLLMENT;
+-----+-----+
| StudentID | CourseID |
+-----+-----+
|          101 |          501 |
|          102 |          501 |
|          103 |          502 |
+-----+-----+
3 rows in set (0.00 sec)
```

Step 7: Verify Using JOIN

```
mysql> SELECT
->     S.StudentID,
->     S.StudentName,
->     C.CourseID,
->     C.CourseName,
->     I.InstructorID,
->     I.InstructorName
-> FROM STUDENT S
-> JOIN ENROLLMENT E
->     ON S.StudentID = E.StudentID
-> JOIN COURSE C
->     ON E.CourseID = C.CourseID
-> JOIN INSTRUCTOR I
->     ON C.InstructorID = I.InstructorID;
```

StudentID	StudentName	CourseID	CourseName	InstructorID	InstructorName
101	Arun	501	DBMS	201	Kumar
102	Bala	501	DBMS	201	Kumar
103	Charan	502	OS	202	Priya

3 rows in set (0.02 sec)

Final 3NF Decomposition

STUDENT(StudentID,StudentName)

COURSE(CourseID,CourseName,InstructorID)

INSTRUCTOR(InstructorID,InstructorName)

ENROLLMENT(StudentID,CourseID)

Result

The insertion, deletion, and update anomalies were identified successfully. The poorly designed college relation was normalized into **3NF**, reducing redundancy and improving data integrity.

Q10. Attribute Closure and Candidate Keys

Aim

To compute the closure of attribute sets for the given functional dependencies and determine the candidate keys of a relation.

Given

The assessment question does **not provide the specific relation or functional dependencies** for Q10. Therefore, the exact candidate keys cannot be determined from the uploaded question paper alone.

Procedure

For an attribute set X , its closure is represented as:

The following steps are used:

1. Start with the attributes in X .
2. Apply the given functional dependencies.
3. Add the newly determined attributes to X^+ .
4. Continue until no new attributes can be added.
5. If X^+ contains all attributes of the relation, X is a **superkey**.
6. If no attribute can be removed from X while retaining the superkey property, X is a **candidate key**.

Example

Consider: $R(A,B,C,D,E)$

Functional dependencies:

$A \rightarrow BC$

$BC \rightarrow D$

$D \rightarrow E$

Calculate A^+

Initially: $A^+ = \{A\}$

Using: $A \rightarrow BC$

we get: $A^+ = \{A, B, C\}$

Using: $BC \rightarrow D$

we get: $A^+ = \{A, B, C, D\}$

Using: $D \rightarrow E$

we get: $A^+ = \{A, B, C, D, E\}$

Therefore: $A^+ = \{A, B, C, D, E\}$

Since A^+ contains all attributes of the relation:

A is a candidate key

MySQL Implementation

MySQL does not have a direct SQL command for calculating functional-dependency closure. The relation can be created and verified as follows:

```
mysql> CREATE TABLE CLOSURE_R (  
->   A INT PRIMARY KEY,  
->   B VARCHAR(20),  
->   C VARCHAR(20),  
->   D VARCHAR(20),  
->   E VARCHAR(20)  
-> );  
Query OK, 0 rows affected (0.04 sec)
```

Insert Sample Data

```
mysql> INSERT INTO CLOSURE_R VALUES  
-> (1, 'B1', 'C1', 'D1', 'E1'),  
-> (2, 'B2', 'C2', 'D2', 'E2'),  
-> (3, 'B3', 'C3', 'D3', 'E3');  
Query OK, 3 rows affected (0.02 sec)  
Records: 3  Duplicates: 0  Warnings: 0
```

Display the Relation

```
mysql> SELECT * FROM CLOSURE_R;  
+----+-----+-----+-----+-----+  
| A | B   | C   | D   | E   |  
+----+-----+-----+-----+-----+  
| 1 | B1  | C1  | D1  | E1  |  
| 2 | B2  | C2  | D2  | E2  |  
| 3 | B3  | C3  | D3  | E3  |  
+----+-----+-----+-----+-----+  
3 rows in set (0.00 sec)
```

Result

The attribute closure was calculated using the given functional dependencies, and A was identified as a candidate key in the example.